データ移行計画書

① 円盤マスタ(*discs*)

旧：SpringBoot版

円盤マスタ(discs)

円盤ID(discs_id)
アーティスト名(artist) 円盤名(title) 補足タイトル(title_sub) 円盤カテゴリ(category) 国名(country) 発売日(release_date)

移行

※CSVファイルにて移行

新：Laravel版

円盤マスタ(discs)	
PK	円盤ID(id)
	管理コード(code)
	アーティスト名(artist)
	円盤名(title)
	補足タイトル(title_sub)
	円盤カテゴリ(category)
	国名(country)
	発売日(release_date)
	作成日時(created_at)
	更新日時(updated_at)

変更点

- PKである*disc_id*をLaravelの命名規則に合わせて*id(自動採番)*に統一
  →Eloquentが自動で主キーを認識し、リレーション設定などを簡略化可能に
- 円盤IDとして振っていた"D000"形式の文字列は既存データとの整合性維持のため
  管理コード*code*として保持
- Laravel標準の*created_at*と*updated_at*を追加

② 形態マスタ(*editions*)

旧:SpringBoot版

形態マスタ(editions)

形態ID(edition_id)
円盤ID(discs_id)
形態名(edition_name)
形態表記名(display_name)
価格(price)
通貨コード(currency)
トラックリスト(tracklist)
特典(benefit)
収録映像内容(video_content)
備考(remarks)
所持フラグ(is_purchased)
ウィッシュリストフラグ(is_wishlist)

移行

※CSVファイルにて移行

新:Laravel版

形態マスタ(editions)

PK	形態ID(id)
FK	円盤ID(disc_id)
	管理コード(code)
	形態名(name)
	形態表記名(display_name)
	価格(price)
	通貨コード(currency)
	トラックリスト(tracklist)
	特典(benefit)
	収録映像内容(video_content)
	備考(remarks)
	作成日時(created_at)
	更新日時(updated_at)
	ソートID(sort_id)

※移行時にレコードの並び順が保持されない課題が発生
→ソートIDを新設し任意の順序で制御できるよう再割り当てを実施

変更点

- PKである*edition_id*をLaravelの命名規則に合わせて*id(自動採番)*に統一
  →Eloquentが自動で主キーを認識し、リレーション設定などを簡略化可能に
- 形態IDとして振っていた"D000-00"形式の文字列は既存データとの整合性維持のため
  管理コード*code*として保持
- Laravel標準の*created_at*と*updated_at*を追加
- *冗長な命名を廃止するため*edition_name*を*name*へ変更
- boolean型である*is_purchased*と*is_wishlist*をユーザーステータステーブルへ移動
  →マスタからユーザー個別の状態を切り離すことで多ユーザー利用に対応し
  データの独立性を高めた

SQL

- 新円盤IDと紐づける為、旧*disc_id*のカラム名を*temp_disc_code*へ変更し
  以下SQLにて実行後削除

```
UPDATE editions
SET disc_id = d.id
FROM discs d
WHERE editions.temp_disc_code = d.code;
```

- CSVファイルに出力時、NULL値が"null"文字列として出力されてしまう為
  NULL値のあるカラムに以下SQLを実行

```
UPDATE editions
SET ○○ = NULL
WHERE ○○ = 'null';
```

③ ランダムアイテムマスタ (*random_items*)

旧：SpringBoot版

ランダムアイテムマスタ (random_item)

アイテムID(item_id)
形態ID(edition_id)
アイテム名(item_type)
メンバー名(member_name)
画像URL(image_url)
登録日時(created_at)

移行

新：Laravel版

ランダムアイテムマスタ(random_items)

PK	アイテムID(id)
FK	円盤ID(disc_id)
FK	形態ID(edition_id)
	メンバー名(member_name)
	アイテム名(item_type)
	作成日時(created_at)
	更新日時(updated_at)

変更点

- PKである*item_id*をLaravelの命名規則に合わせて*id(自動採番)*に統一
  →Eloquentが自動で主キーを認識し、リレーション設定などを簡略化可能に

- Laravel標準の*updated_at*を追加
  *created_at*は論理名を揃え一貫性を高めた

- *image_url*をランダムユーザーステータステーブルへ移動
  →マスタからユーザー個別の状態を切り離すことで多ユーザー利用に対応し
  データの独立性を高め、ランダム管理ページの仕様変更へ対応

- マスタテーブルの物理名をLaravel標準の複数形へ統一

- 上位マスタである円盤マスタ*discs*との直接的なリレーションを追加
  →円盤＞形態＞ランダム商品(特典)という階層構造をデータベースレベルで担保し
  データの参照整合性を強化

SQL

- 新形態IDと紐づける為、旧*edition_id*のカラム名を*temp_edition_code*へ変更し
  以下SQLにて実行後削除

```
UPDATE random_items
SET edition_id = e.id
FROM editions e
WHERE random_items.temp_edition_code = e.code;
```

- 円盤IDと紐づける為、以下のSQLを実行
  形態情報を介して親の*disc_id*を取得　※新形態IDと紐づけ完了後実行すること

```
UPDATE random_items
SET disc_id = e.disc_id
FROM editions e
WHERE random_items.edition_id = e.id;
```

④ ユーザーマスタ(*users*)

Laravel版:新規作成

ユーザーマスタ(users)	
PK	ユーザーID(id)
	ユーザー名(name)
	メールアドレス(email)
	パスワード(password)
	アイコン画像URL(icon_url)
	ログイン保持トークン(remember_token)
	作成日時(created_at)
	更新日時(updated_at)

追加理由

・SpringBoot版では全ユーザー共通であったが、Laravel版ではユーザー別管理機能を導入する為

仕様

・パスワード:Laravelの標準機能に合わせハッシュ化して保存

・ログイン保持トークン:Laravel標準機能でログイン認証時にトークンが発行・照合される仕組みを利用

⑤ ユーザーステータス (*user_status*)

Laravel版:新規作成

ユーザーステータス(user_status)	
PK	ステータスID(id)
FK	形態ID(edition_id)
FK	ユーザーID(user_id)
	所持フラグ(is_purchased)
	ウィッシュリストフラグ(is_wishlist)
	作成日時(created_at)
	更新日時(updated_at)

追加理由

・ユーザー別管理機能追加に伴い、形態マスタにあったboolean型カラム個別管理する為

仕様

・所持フラグ:所持未所持のフラグ管理用

・ウィッシュリストフラグ:直近でほしい、今度買いたいなの部類にある形態を管理する用

⑥ ランダムユーザーステータス (*user_randoms*)

Laravel版：新規作成

ランダムユーザーステータス(user_randoms)	
PK	ランダムステータスID(id)
FK	アイテムID(item_id)
FK	ユーザーID(user_id)
	画像URL(image_url)
	作成日時(created_at)
	更新日時(updated_at)

追加理由

・ユーザー別管理機能追加と、ランダム管理ページの仕様変更に対応する為

仕様

・ランダムアイテムマスタで作った大枠にユーザー別で所持ランダム品の画像をアップロードしていく形式

拡張性

・備考欄を作り、何個何枚持っているかを入力できるようにする

・無限収集対象のフラグ等を付ける

⑦ 問い合わせマスタ (*contacts*)

Laravel版：新規作成

問い合わせマスタ(contacts)	
PK	問い合わせID(id)
	氏名(name)
	メールアドレス(email)
	件名(subject)
	本文(body)
	作成日時(created_at)
	更新日時(updated_at)

追加理由

・ダミーではない送信完了画面を導入するため

仕様

・ユーザーでないゲスト状態でも送信可能